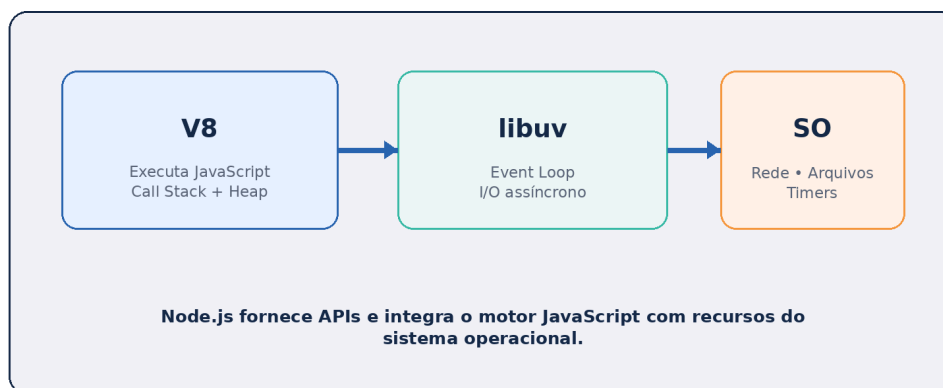




Node.js — Projeto Guiado

Node.js: JavaScript executando em um runtime de servidor



MÓDULO 1 — FUNDAMENTOS DE BACKEND, JAVASCRIPT MODERNO E NODE.JS

Conceitos, sintaxe, organização profissional e práticas de desenvolvimento

Joinville - 2026

Prof. Carlos Uchôa e Prof. Sestito

SUMÁRIO

1 INTRODUÇÃO	3	15 JSON: FORMATO DE TROCA DE DADOS	15
2 OBJETIVOS DO MÓDULO	3	16 MÓDULOS: ORGANIZANDO RESPONSABILIDADES	16
3 O QUE É BACKEND?	4	17 ERROS, EXCEÇÕES E FAIL FAST	17
4 JAVASCRIPT, NODE.JS E O RUNTIME	5	18 PROMISES E ASYNC/AWAIT — PRIMEIRO CONTATO	18
5 PREPARANDO O PROJETO DO MÓDULO	6	19 PROCESS, ARGUMENTOS E VARIÁVEIS DE AMBIENTE	19
6 VARIÁVEIS, CONSTANTES E ESCOPO	7	20 BOAS PRÁTICAS DE CÓDIGO PARA BACKEND	19
7 TIPOS, VALORES E COMPARAÇÕES	8	21 VALIDAÇÃO NAS FRONTEIRAS	20
8 STRINGS, TEMPLATE LITERALS E CONVERSÕES	9	22 ESTRUTURA INICIAL DO BACKEND	21
9 OPERADORES MODERNOS ÚTEIS NO BACKEND	10	23 MINIPROJETO GUIADO — CATÁLOGO DE SERVIÇOS	23
10 CONTROLE DE FLUXO E EARLY RETURN	11	24 ANTI-PADRÕES QUE DEVEM SER EVITADOS DESDE O INÍCIO	25
11 FUNÇÕES: A UNIDADE DE COMPORTAMENTO	12	25 CHECKLIST DE DOMÍNIO DA SINTAXE	25
12 ESCOPO, CLOSURES E ESTADO	13	26 ATIVIDADES DE FIXAÇÃO DO MÓDULO	26
13 ARRAYS E PROGRAMAÇÃO ORIENTADA A COLEÇÕES	13	27 CONSIDERAÇÕES FINAIS	27
14 OBJETOS, DESTRUCTURING E SPREAD	14	28 REFERÊNCIAS	27

1 INTRODUÇÃO

O **Módulo 0** estabeleceu um ambiente de desenvolvimento reproduzível, versionado e próximo de um cenário profissional. A partir deste ponto, o foco deixa de ser apenas preparar a máquina e passa a ser compreender o que estamos construindo, como o Node.js executa nosso código e como organizar um backend para que ele permaneça legível, testável e evolutivo.

Este módulo revisa a base conceitual de backend e a sintaxe moderna de JavaScript utilizada dentro do Node.js. O objetivo não é decorar recursos da linguagem. Cada conceito será associado a uma situação de desenvolvimento: receber dados, validar informações, executar regras, organizar módulos, tratar falhas e preparar uma estrutura que possa crescer até a API de Gestão de Serviços e Agendamentos prevista no projeto guiado.

Princípio do curso

Continuaremos seguindo a sequência problema → conceito → ferramenta. Uma abstração só será adicionada quando existir um problema concreto que justifique sua presença.

O mapa da formação define que o projeto deve evoluir de forma contínua, aproximando conceito, aplicação e situação profissional; também prevê JavaScript moderno, runtime Node.js e módulos antes de avançarmos para HTTP e frameworks. Este Módulo 1 materializa exatamente essa etapa.

2 OBJETIVOS DO MÓDULO

Ao final deste módulo, o acadêmico deverá ser capaz de:

- Explicar a responsabilidade de um backend e diferenciá-lo de frontend, banco de dados, API e runtime.
- Explicar a relação JavaScript → Node.js e identificar o papel do V8, das APIs do Node e do sistema operacional.
- Utilizar corretamente const, let, tipos, operadores, estruturas condicionais, funções, arrays, objetos e módulos.
- Aplicar destructuring, spread/rest, template literals, optional chaining e nullish coalescing em código de backend.
- Organizar código em ES Modules com import/export e utilizar módulos nativos com o prefixo node:.
- Tratar falhas com Error, throw, try/catch e reconhecer a diferença entre erro técnico e regra de negócio.
- Utilizar Promises e async/await em exemplos simples, preparando-se para o estudo aprofundado do Event Loop.
- Criar uma estrutura inicial de projeto Node.js que separe entrada, regra de aplicação e acesso a dados.
- Aplicar boas práticas de nomenclatura, funções pequenas, early return, imutabilidade pragmática e validação nas fronteiras.

- Concluir um mini-backend sem HTTP, usando memória como persistência temporária e estrutura pronta para evoluir.

Fluxo essencial de uma aplicação backend

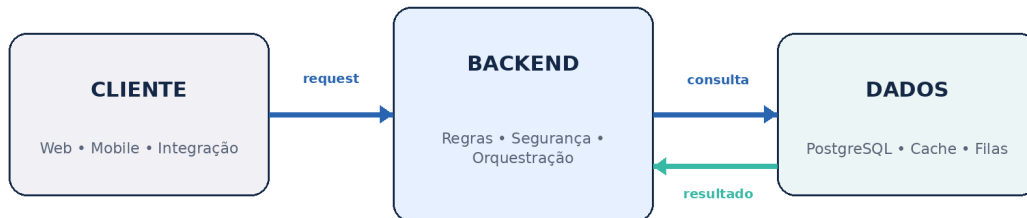


Figura 1 — Visão geral do papel do backend entre clientes e mecanismos de dados.

3 O QUE É BACKEND?

Backend é a parte de um sistema responsável por processar solicitações, aplicar regras de negócio, proteger recursos, integrar serviços e coordenar o acesso aos dados. Em uma aplicação web ou mobile, o usuário normalmente não interage diretamente com o backend; ele interage com uma interface que envia solicitações para o servidor.

Uma forma simples de visualizar o fluxo é: cliente → backend → dados/serviços externos → backend → cliente. O backend recebe uma entrada, decide o que pode e deve ser feito e devolve uma resposta adequada.

3.1 Backend não é banco de dados

O banco armazena e consulta dados. O backend decide como esses dados podem ser utilizados. Uma regra como “um profissional não pode possuir dois agendamentos no mesmo horário” não deve existir apenas como um detalhe de tela. Ela pertence ao comportamento do sistema e precisa ser garantida no servidor.

3.2 Backend não é API

API é uma interface de comunicação. Um backend pode oferecer uma API HTTP, consumir filas, processar tarefas agendadas ou executar comandos de terminal. A API é uma porta de entrada; o backend é o conjunto de responsabilidades por trás dela.

3.3 Responsabilidades típicas

Responsabilidade	Exemplo no projeto guiado
Validação de entrada	Garantir que nome, duração e preço de um serviço tenham formato aceitável.
Regra de negócio	Impedir conflito de horários para o mesmo profissional.
Autenticação	Identificar o usuário que está acessando a aplicação.
Autorização	Definir se esse usuário pode cadastrar profissionais ou visualizar auditoria.
Persistência	Salvar pessoas, serviços e agendamentos no PostgreSQL.
Integração	Enviar e-mail, publicar evento ou consultar um serviço externo.
Observabilidade	Registrar logs e métricas para diagnosticar falhas.
Segurança	Limitar entrada, proteger credenciais e reduzir superfície de ataque.

Boa prática profissional

Validação deve ocorrer o mais cedo possível na entrada de dados. Depois dela, a aplicação ainda precisa executar validações semânticas e regras de negócio.

EXERCÍCIO DE FIXAÇÃO — Backend e responsabilidades

1. Explique com suas palavras a diferença entre backend, API e banco de dados.
2. No domínio de agendamentos, escreva três regras que devem obrigatoriamente ser garantidas no backend.
3. Classifique cada item como validação sintática ou regra de negócio: e-mail sem @; preço negativo; conflito de horário; CPF com quantidade errada de dígitos; serviço desativado sendo agendado.

Desafio profissional: Escolha um sistema que você já utiliza e desenhe, em até seis blocos, o caminho de uma ação do usuário até a persistência dos dados.

4 JAVASCRIPT, NODE.JS E O RUNTIME

JavaScript é a linguagem. Node.js é um runtime que permite executar JavaScript fora do navegador, oferecendo integração com arquivos, rede, processos, variáveis de ambiente e outros recursos do sistema operacional. Essa distinção é fundamental: Node.js não é uma linguagem e também não é um framework web.

Node.js: JavaScript executando em um runtime de servidor

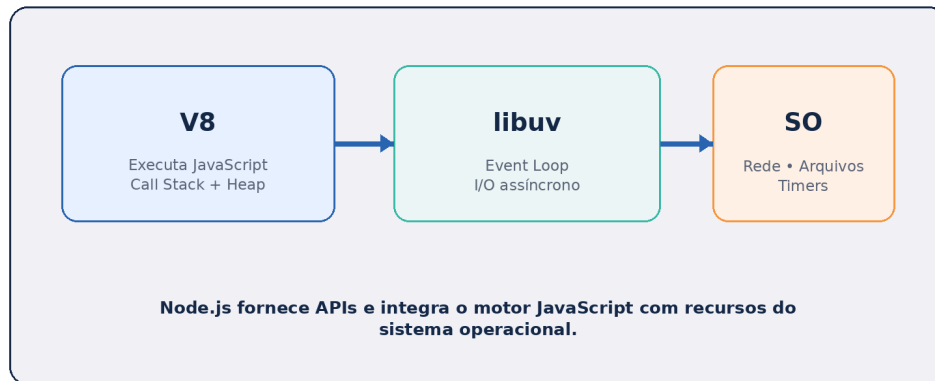


Figura 2 — Visão didática do runtime Node.js: V8, libuv e integração com o sistema operacional.

4.1 V8, APIs do Node e libuv

O V8 compila e executa JavaScript. O Node.js adiciona APIs próprias, como fs, path, http, process e timers. A libuv fornece a base do Event Loop e de várias operações assíncronas de I/O. O funcionamento detalhado do Event Loop será aprofundado no próximo módulo; neste momento, precisamos reconhecer que operações de rede e arquivo não devem ser tratadas como se fossem cálculos puramente síncronos.

4.2 Executando arquivos

```
node src/app.js
```

O comando node inicia o runtime e entrega o arquivo informado para execução. No projeto deste módulo utilizaremos ES Modules, configurando "type": "module" no package.json.

Conceito importante

“Sintaxe do Node.js” é, em grande parte, sintaxe de JavaScript executada no runtime Node.js. O que muda são as APIs disponíveis no ambiente e o modelo de módulos/configuração utilizado pelo projeto.

EXERCÍCIO DE FIXAÇÃO — Runtime

1. Explique por que console.log pode existir no navegador e no Node.js, embora os dois ambientes sejam diferentes.
2. Pesquise no REPL o resultado de process.version e process.platform e explique o que cada propriedade representa.

3. Liste três recursos que fazem sentido no Node.js mas não são responsabilidades típicas de uma página no navegador.

5 PREPARANDO O PROJETO DO MÓDULO

Criaremos uma continuação do repositório iniciado no Módulo 0. Se o grupo estiver utilizando outro nome, mantenha a estrutura equivalente. O projeto será executado inicialmente em JavaScript moderno para revisar a linguagem sem esconder seus conceitos atrás do TypeScript. A migração para TypeScript ocorrerá de forma consciente posteriormente.

```
mkdir node-profissional
cd node-profissional
npm init -y
mkdir src
```

No package.json, adicione:

```
{
  "name": "node-profissional",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "node --watch src/app.js",
    "start": "node src/app.js"
  }
}
```

O modo --watch reinicia a execução quando arquivos utilizados pela aplicação são alterados. Para este módulo ele é suficiente e reduz a dependência de ferramentas adicionais.

Boa prática de projeto

Scripts importantes devem existir no package.json. Assim, a equipe executa npm run dev e npm start sem precisar memorizar comandos específicos de cada máquina.

6 VARIÁVEIS, CONSTANTES E ESCOPO

6.1 const, let e var

Em código moderno, const deve ser a escolha padrão. Use let quando a referência realmente precisar receber outro valor. Evite var em código novo, pois seu comportamento de escopo e hoisting é mais propenso a ambiguidades.

```
const serviceName = "Consulta";
let availableSlots = 10;

availableSlots = 9;

// serviceName = "Retorno"; // TypeError: reatribuição não permitida
```

const impede atribuir à variável, mas não torna objetos automaticamente imutáveis:

```
const service = { name: "Consulta", active: true };
service.active = false; // permitido

// service = {}; // não permitido
```

6.2 Escopo de bloco

```
if (true) {
  const message = "visível somente neste bloco";
  let counter = 1;
}

// console.log(message); // ReferenceError
```

Boa prática profissional

Prefira o menor escopo possível. Quanto menos lugares puderem alterar um valor, menor a carga mental para entender o comportamento do sistema.

EXERCÍCIO DE FIXAÇÃO — Variáveis e escopo

1. Reescreva um exemplo com var utilizando const/let de forma adequada.
2. Crie uma constante service com name, durationMinutes e active. Depois altere apenas active.
3. Explique por que const service = {} não torna service imutável.

7 TIPOS, VALORES E COMPARAÇÕES

7.1 Tipos primitivos

Tipo	Exemplo	Uso comum em backend
string	"Consulta"	nomes, códigos, tokens, mensagens
number	90.5	preços, duração, contadores
boolean	true	flags e estados simples
undefined	undefined	valor ainda não definido
null	null	ausência intencional de valor
bigint	9007199254740993n	inteiros maiores que o limite seguro
symbol	Symbol("id")	chaves únicas em casos específicos


```
console.log(typeof "Node"); // string
console.log(typeof 42); // number
console.log(typeof true); // boolean
console.log(typeof undefined); // undefined
console.log(typeof null); // object (peculiaridade histórica)
```

7.2 === versus ==

Em backend profissional, prefira comparação estrita. O operador == realiza coerção de tipos e pode produzir resultados inesperados.

```
0 == "0"; // true
0 === "0"; // false

false == 0; // true
false === 0; // false
```

7.3 Truthy e falsy

Valores como false, 0, "", null, undefined e NaN são falsy. Esse recurso é útil, mas pode esconder regras quando zero ou string vazia forem valores válidos.

```
const duration = 0;

if (!duration) {
  console.log("0 também entra aqui");
}
```

Atenção

Não confunda “ausente” com “falsy”. Em validações de entrada, compare explicitamente quando 0, false ou string vazia poderem possuir significado de domínio.

EXERCÍCIO DE FIXAÇÃO — Tipos e comparações

1. Execute cinco comparações com == e === e explique os resultados.
2. Crie uma validação que aceite o valor 0 para discountPercentage, mas rejeite null e undefined.
3. Use Number.isNaN para verificar o resultado de Number("abc").

8 STRINGS, TEMPLATE LITERALS E CONVERSÕES

```
const professional = "Ana";
const hour = "14:30";

const message = `Agendamento com ${professional} às ${hour}.`;
console.log(message);
```

Templates literals reduzem concatenações e tornam mensagens de log, erros e respostas mais legíveis. Conversões devem ser explícitas nas fronteiras do sistema: dados de `process.argv`, query strings e variáveis de ambiente normalmente chegam como texto.

```
const rawDuration = "45";
const duration = Number(rawDuration);

if (!Number.isFinite(duration)) {
  throw new Error("Duração inválida");
}
```

EXERCÍCIO DE FIXAÇÃO — Strings e conversão

1. Monte uma mensagem usando template literal com nome, preço e duração de um serviço.
2. Converta "19.90" para number e formate o resultado com duas casas decimais.
3. Demonstre por que `parseInt("10min", 10)` e `Number("10min")` produzem comportamentos diferentes.

9 OPERADORES MODERNOS ÚTEIS NO BACKEND

9.1 Optional chaining — ?.

```
const person = { address: null };

const city = person.address?.city;
console.log(city); // undefined
```

9.2 Nullish coalescing — ??

```
const pageSizeFromInput = 0;
const pageSize = pageSizeFromInput ?? 20;

console.log(pageSize); // 0
```

Diferentemente de `||`, o operador `??` só utiliza o valor alternativo quando o lado esquerdo é `null` ou `undefined`. Isso evita substituir `0`, `false` ou `""` inadvertidamente.

9.3 Operador ternário

```
const status = service.active ? "ACTIVE" : "INACTIVE";
```

Regra de legibilidade

Ternários curtos são úteis. Ternários aninhados, especialmente com regras de negócio, normalmente devem ser substituídos por if/else ou funções bem nomeadas.

EXERCÍCIO DE FIXAÇÃO — Operadores

1. Use optional chaining para acessar `person.contact.phone` sem gerar erro quando `contact` não existir.
2. Compare o resultado de `0 || 20` e `0 ?? 20`.
3. Reescreva um ternário aninhado usando if/else e avalie qual versão é mais legível.

10 CONTROLE DE FLUXO E EARLY RETURN

```
function calculatePrice(price, quantity) {
  if (quantity <= 0) {
    throw new Error("Quantidade deve ser maior que zero");
  }

  if (price < 0) {
    throw new Error("Preço não pode ser negativo");
  }

  return price * quantity;
}
```

Early return reduz níveis de indentação e deixa o caminho principal mais evidente. Em funções de backend, validar condições inválidas no início costuma tornar o código mais simples do que criar grandes árvores de if/else.

10.1 switch

```
function getStatusLabel(status) {
  switch (status) {
    case "SCHEDULED":
      return "Agendado";
    case "CANCELED":
      return "Cancelado";
    default:
      return "Desconhecido";
  }
}
```

EXERCÍCIO DE FIXAÇÃO — Controle de fluxo

1. Crie uma função `canSchedule` que rejeite serviço inativo e duração menor ou igual a zero usando `early return`.
2. Implemente um `switch` para os perfis `ADMIN`, `PROFESSIONAL` e `CUSTOMER`.
3. Refatore uma sequência de três `if` aninhados para reduzir a profundidade.

11 FUNÇÕES: A UNIDADE DE COMPORTAMENTO

11.1 Declaração, expressão e arrow function

```
function sum(a, b) {  
  return a + b;  
}  
  
const subtract = function (a, b) {  
  return a - b;  
};  
  
const multiply = (a, b) => a * b;
```

No backend, mais importante que escolher um único estilo é escrever funções com responsabilidade clara, nomes descritivos, entradas compreensíveis e efeitos colaterais controlados.

11.2 Parâmetros padrão e rest

```
function listServices(services, limit = 20) {  
  return services.slice(0, limit);  
}  
  
function sumValues(...values) {  
  return values.reduce((total, value) => total + value, 0);  
}
```

11.3 Função pura e efeito colateral

```
// função pura  
function applyDiscount(price, percentage) {  
  return price - price * (percentage / 100);  
}  
  
// possui efeito colateral  
function logServiceCreated(service) {  
  console.log("service_created", service.id);  
}
```

Boa prática profissional

Separar cálculo puro de I/O facilita testes. Uma regra de preço não precisa conhecer console, banco de dados ou HTTP.

EXERCÍCIO DE FIXAÇÃO — Funções

1. Crie uma função `calculateEndTime(startMinutes, durationMinutes)` sem acessar variáveis externas.
2. Crie uma função `formatService` que retorna uma string e não altere o objeto recebido.
3. Explique qual das duas funções seria mais fácil testar e por quê.

12 ESCOPO, CLOSURES E ESTADO

Uma closure ocorre quando uma função mantém acesso ao escopo em que foi criada. Isso pode ser útil para encapsular o estado, mas exige cuidado em aplicações long-running, porque estado mantido em memória vive enquanto o processo estiver ativo.

```
function createCounter() {
  let value = 0;

  return function increment() {
    value += 1;
    return value;
  };
}

const nextId = createCounter();
console.log(nextId()); // 1
console.log(nextId()); // 2
```

Pensamento de produção

Um contador em memória não é confiável como identificador distribuído. Se o processo reiniciar ou existirem duas instâncias da aplicação, cada uma terá seu próprio estado. Esse exemplo serve para aprender closure, não para substituir o banco.

EXERCÍCIO DE FIXAÇÃO — Closures e estado

1. Implemente `createSequence(start)` que retorne uma função geradora de números.
2. Explique o que acontece com o estado quando o processo Node.js é encerrado.
3. Explique por que duas instâncias do backend não compartilhariam esse contador.

13 ARRAYS E PROGRAMAÇÃO ORIENTADA A COLEÇÕES

APIs frequentemente recebem, filtram, transformam e agregam coleções. Por isso, dominar os métodos de Array reduz loops imperativos e torna intenções mais explícitas.

Método	Retorno	Uso típico
map	novo array	transformar registros
filter	novo array	selecionar itens
find	um item ou undefined	buscar primeiro correspondente
some	boolean	verificar se existe ocorrência
every	boolean	validar todos os itens
reduce	valor acumulado	somar/agrupar/transformar
sort	array ordenado (muta!)	ordenar; preferir toSorted quando disponível

```
const services = [
  { id: 1, name: "Consulta", active: true, price: 150 },
  { id: 2, name: "Retorno", active: false, price: 80 },
  { id: 3, name: "Avaliação", active: true, price: 120 },
];

const activeServices = services.filter((service) => service.active);
const names = activeServices.map((service) => service.name);
const total = activeServices.reduce((sum, service) => sum + service.price, 0);
```

13.1 Evitar mutação acidental

```
const original = [3, 1, 2];
const sorted = original.toSorted((a, b) => a - b);

console.log(original); // [3, 1, 2]
console.log(sorted); // [1, 2, 3]
```

EXERCÍCIO DE FIXAÇÃO — Arrays

1. Dado um array de serviços, retorne somente os ativos com preço maior que 100.
2. Gere um novo array contendo apenas { id, name }.
3. Verifique com some se existe algum serviço inativo.
4. Calcule o preço médio dos serviços usando reduce.

Desafio profissional: Implemente uma função `paginate(items, page, pageSize)` e teste com pelo menos 25 itens.

14 OBJETOS, DESTRUCTURING E SPREAD

```
const service = {
  id: 10,
  name: "Consulta",
  durationMinutes: 45,
  active: true,
};
```

```
const { id, name, ...details } = service;  
const updated = { ...service, active: false };
```

O spread permite criar novas representações sem modificar diretamente o objeto original. Isso é útil ao montar DTOs, atualizar estado em memória e preparar dados para persistência.

14.1 Property shorthand

```
const name = "Consulta";  
const durationMinutes = 45;  
  
const service = { name, durationMinutes };
```

14.2 Object.entries

```
for (const [key, value] of Object.entries(service)) {  
  console.log(key, value);  
}
```

EXERCÍCIO DE FIXAÇÃO — Objetos

1. Extraia name e price de um objeto service usando destructuring.
2. Crie um novo objeto com spread alterando apenas price.
3. Remova uma propriedade interna chamada internalNote usando rest destructuring antes de devolver o objeto ao cliente.

15 JSON: FORMATO DE TROCA DE DADOS

JSON é um texto estruturado. Objetos JavaScript e JSON não são a mesma coisa. Ao receber conteúdo serializado, precisamos converter texto para objeto; ao enviar ou salvar representação textual, fazemos o caminho inverso.

```
const payload = JSON.stringify({ name: "Consulta", price: 150 });  
console.log(payload);  
  
const data = JSON.parse(payload);  
console.log(data.name);
```

Atenção

JSON.parse pode lançar SyntaxError. Dados externos precisam ser tratados como não confiáveis e erros de parsing devem ser capturados na fronteira adequada.

EXERCÍCIO DE FIXAÇÃO — JSON

1. Converta um objeto com service e professional para JSON.
2. Tente executar JSON.parse com um texto inválido e capture o erro.
3. Explique por que funções e undefined não são representados normalmente em JSON.

16 MÓDULOS: ORGANIZANDO RESPONSABILIDADES

Node.js suporta CommonJS e ECMAScript Modules. Nesta formação adotaremos ES Modules. O uso de import/export torna as dependências explícitas e aproxima a sintaxe utilizada do padrão oficial da linguagem.

ES Modules: dependências explícitas entre arquivos



Quanto mais explícitas as dependências, mais fácil compreender, testar e substituir partes do sistema.

Figura 3 — Dependências explícitas usando ES Modules.

```

// src/modules/services/validator.js
export function validateServiceInput(input) {
  if (!input.name?.trim()) {
    throw new Error("Nome é obrigatório");
  }
}

// src/modules/services/service.js
import { validateServiceInput } from './validator.js';

export function createService(input) {
  validateServiceInput(input);
  return { ...input, active: true };
}
  
```


16.1 Módulos nativos com node:

```
import path from "node:path";
import { readFile } from "node:fs/promises";
```

O prefixo node: deixa explícito que o módulo pertence ao runtime e evita confusão com dependências instaladas pelo gerenciador de pacotes.

Padrão da disciplina

Use extensão .js nos imports relativos do projeto ESM. Não esconda dependências em variáveis globais. Um arquivo deve importar o que utiliza.

EXERCÍCIO DE FIXAÇÃO — ES Modules

1. Crie math.js exportando duas funções e importe-as em app.js.
2. Crie service-validator.js e service.js, deixando a validação em um módulo separado.
3. Use node:path para obter a extensão de um caminho informado.

17 ERROS, EXCEÇÕES E FAIL FAST

Falhas fazem parte de sistemas reais. O objetivo não é eliminar toda possibilidade de erro, mas detectar condições inválidas, comunicar a causa corretamente e impedir que o sistema continue executando com estado incoerente.

```
function validateDuration(durationMinutes) {
  if (!Number.isInteger(durationMinutes) || durationMinutes <= 0) {
    throw new Error("durationMinutes deve ser inteiro positivo");
  }
}

try {
  validateDuration(-10);
} catch (error) {
  console.error(error.message);
}
```

17.1 Erro técnico versus erro de negócio

Categoria	Exemplo	Tratamento futuro
Entrada inválida	durationMinutes = -10	Resposta 400 / erro de validação
Regra de negócio	horário já ocupado	Resposta 409 / conflito
Infraestrutura	banco indisponível	Log, retry quando aplicável, resposta 5xx
Programação	acesso a propriedade inexistente por bug	Diagnóstico, teste e correção

Boa prática profissional

Não use try/catch para esconder falhas. Capturar uma exceção só faz sentido quando a camada atual sabe como transformar, registrar, compensar ou propagar o problema.

EXERCÍCIO DE FIXAÇÃO — Tratamento de erros

1. Crie validatePrice que lance Error quando o preço for inválido.
2. Capture esse erro no app.js e imprima uma mensagem amigável.
3. Dê um exemplo de erro que não deveria ser “engolido” silenciosamente.

18 PROMISES E ASYNC/AWAIT — PRIMEIRO CONTATO

As operações de rede, banco e arquivos frequentemente são assíncronas. Uma Promise representa um resultado que estará disponível no futuro. async/await fornece uma sintaxe mais legível para trabalhar com Promises, mas não transforma operações de I/O em código síncrono.

```
import { readFile } from "node:fs/promises";

async function loadConfig() {
  const content = await readFile("./config.json", "utf8");
  return JSON.parse(content);
}

try {
  const config = await loadConfig();
  console.log(config);
} catch (error) {
  console.error("Falha ao carregar configuração", error.message);
}
```

Top-level await é permitido em ES Modules. Mesmo assim, funções async bem definidas costumam facilitar composição, tratamento de erros e testes.

Próximo módulo

Event Loop, Call Stack, filas, microtasks, I/O e concorrência serão estudados em profundidade. Aqui queremos dominar a sintaxe e reconhecer quando uma operação retorna Promise.

EXERCÍCIO DE FIXAÇÃO — Async/await

1. Use node:fs/promises para ler um arquivo services.json.
2. Trate o caso de arquivo inexistente.
3. Crie uma função async que aguarde Promise.resolve(42) e devolva o dobro do valor.

19 PROCESS, ARGUMENTOS E VARIÁVEIS DE AMBIENTE

19.1 process.argv

```
// node src/app.js "Consulta" 45
const [, , name, rawDuration] = process.argv;
const durationMinutes = Number(rawDuration);

console.log({ name, durationMinutes });
```

19.2 process.env

```
const port = Number(process.env.PORT ?? 3000);
console.log(`Aplicação configurada para a porta ${port}`);
```

Segurança

Variáveis de ambiente são úteis para configuração, mas não devem ser consideradas automaticamente seguras. Segredos não devem ser impressos em logs nem versionados em arquivos .env reais.

EXERCÍCIO DE FIXAÇÃO — Processo e configuração

1. Crie um script que receba name e durationMinutes via linha de comando.
2. Leia APP_ENV de process.env e use "development" como padrão.
3. Explique por que process.env.PORT é string e precisa ser convertido.

20 BOAS PRÁTICAS DE CÓDIGO PARA BACKEND

20.1 Nomes que revelam intenção

```
// ruim
function calc(a, b) { return a * b; }

// melhor
function calculateServiceTotal(unitPrice, quantity) {
  return unitPrice * quantity;
}
```

20.2 Funções pequenas e coesas

Uma função deve possuir uma responsabilidade principal. Se o mesmo bloco valida entrada, acessa banco, formata mensagem e envia e-mail, ele tende a ser difícil de testar e alterar.

20.3 Evitar valores mágicos

```
const MIN_SERVICE_DURATION_MINUTES = 15;

if (durationMinutes < MIN_SERVICE_DURATION_MINUTES) {
  throw new Error("Duração abaixo do mínimo permitido");
}
```

20.4 Comentários explicam o porquê

Comentários não devem compensar nomes ruins. Use-os para registrar decisão, restrição externa, workaround ou motivo que não seja evidente pelo código.

20.5 Duplicação versus abstração prematura

Nem toda repetição precisa ser abstraída imediatamente. Duas linhas parecidas podem representar conceitos diferentes. Espere compreender a regra antes de criar helpers genéricos difíceis de manter.

EXERCÍCIO DE FIXAÇÃO — Clean code pragmático

1. Renomeie variáveis a, b e x em uma função de cálculo de comissão.
2. Extraia uma validação repetida para uma função com nome de domínio.
3. Identifique um comentário que apenas repete o código e substitua-o por um nome melhor.

21 VALIDAÇÃO NAS FRONTEIRAS

Entrada externa: validar cedo, regra de negócio depois



Figura 4 — Validação sintática e regra de negócio ocupam etapas diferentes do fluxo.

Dados vindos de HTTP, CLI, arquivo, fila, webhook ou integração externa não devem ser considerados confiáveis. Primeiro validamos formato e tipo; depois aplicamos regras relacionadas ao domínio. O OWASP recomenda validar entradas tão cedo quanto possível e combinar validação sintática com validação semântica.

```
export function validateServiceInput(input) {
  const name = input?.name?.trim();
  const durationMinutes = Number(input?.durationMinutes);

  if (!name) {
    throw new Error("Nome é obrigatório");
  }

  if (!Number.isInteger(durationMinutes) || durationMinutes <= 0) {
    throw new Error("Duração inválida");
  }

  return { name, durationMinutes };
}
```

Neste momento a validação é manual para que o acadêmico compreenda o problema. Quando a quantidade de campos e formatos crescer, introduziremos validação por schema com uma ferramenta específica.

EXERCÍCIO DE FIXAÇÃO — Validação

1. Valide name, price e durationMinutes de um serviço.
2. Diferencie no código: formato inválido versus “serviço com este nome já existe”.
3. Crie pelo menos cinco casos de entrada inválida e verifique se todos falham de forma previsível.

22 ESTRUTURA INICIAL DO BACKEND

Chegamos ao momento de organizar o código para que novas funcionalidades não transformem app.js em um único arquivo. A estrutura deve surgir de responsabilidades reais e continuar pequena.

Estrutura incremental: separar responsabilidades antes de escalar o projeto



Figura 5 — Separação incremental entre entrada, serviço de aplicação, repositório e dados.

```
src/  
├── app.js  
├── modules/  
│   └── services/  
│       ├── service-validator.js  
│       ├── service-service.js  
│       └── service-repository.js  
└── shared/  
    ├── errors/  
    └── application-error.js
```

Essa estrutura ainda não é a arquitetura final do curso. Ela será expandida conforme surgirem HTTP, banco, autenticação, testes e observabilidade. O importante é que o acadêmico saiba explicar a razão de cada arquivo.

22.1 Repository em memória

```
// src/modules/services/service-repository.js  
const services = [];  
  
export function save(service) {  
  services.push(service);  
  return service;  
}  
  
export function findByName(name) {  
  return services.find((service) => service.name === name);  
}  
  
export function findAll() {  
  return [...services];  
}
```

22.2 Service com regra de aplicação

```
// src/modules/services/service-service.js  
import { findByName, save } from "../service-repository.js";  
import { validateServiceInput } from "../service-validator.js";  
  
export function createService(input) {  
  const data = validateServiceInput(input);  
  
  if (findByName(data.name)) {  
    throw new Error("Serviço já cadastrado");  
  }  
  
  const service = {  
    id: crypto.randomUUID(),  
    ...data,  
    active: true,  
    createdAt: new Date().toISOString(),  
  };  
  return save(service);  
}
```

```
};  
return save(service);  
}
```

No Node.js moderno, crypto é disponibilizado globalmente em vários contextos, mas para tornar a dependência explícita e didática também podemos importar randomUUID de node:crypto:

```
import { randomUUID } from "node:crypto";
```

22.3 Entrada da aplicação

```
// src/app.js  
import { createService } from "../modules/services/service-service.js";  
  
try {  
  const service = createService({  
    name: "Consulta inicial",  
    durationMinutes: 45,  
  });  
  
  console.log("Serviço criado:", service);  
} catch (error) {  
  console.error("Não foi possível criar o serviço:", error.message);  
}
```

Arquitetura pedagógica

app.js conhece o caso de uso; o caso de uso conhece validação e repositório; o repositório conhece a forma atual de guardar dados. Quando o PostgreSQL chegar, queremos substituir a persistência sem reescrever toda a regra.

23 MINIPROJETO GUIADO — CATÁLOGO DE SERVIÇOS

Agora consolidaremos o módulo construindo a primeira fatia real do projeto de Gestão de Serviços e Agendamentos. Ainda não existe HTTP nem banco de dados: o foco é linguagem, responsabilidade e organização.

23.1 Requisitos

- Cadastrar serviço com id, name, durationMinutes, price, active e createdAt.
- Não permitir nome vazio.
- Não permitir durationMinutes menor ou igual a zero.
- Não permitir price negativo.
- Não permitir dois serviços com o mesmo nome, ignorando diferenças de maiúsculas/minúsculas.

- Listar todos os serviços.
- Listar somente serviços ativos.
- Desativar um serviço existente.
- Calcular o valor médio dos serviços ativos.
- Não permitir que app.js acesse diretamente o array do repositório.

23.2 Etapa 1 — normalizar dados

```
function normalizeName(value) {  
  return value.trim().replace(/\s+/g, " ");  
}
```

23.3 Etapa 2 — regra de unicidade

```
const normalizedName = data.name.toLocaleLowerCase("pt-BR");  
  
const exists = findAll().some(  
  (service) => service.name.toLocaleLowerCase("pt-BR") === normalizedName,  
);
```

23.4 Etapa 3 — listagem sem expor estado interno

```
export function findAll() {  
  return services.map((service) => ({ ...service }));  
}
```

Discussão de arquitetura

Copiar objetos rasos reduz as alterações acidentais do estado interno, mas não resolve estruturas profundamente aninhadas. O objetivo aqui é reconhecer o problema de compartilhamento de referências.

23.5 Etapa 4 — relatório

```
export function getActiveServicesSummary() {  
  const active = findAll().filter((service) => service.active);  
  
  const totalPrice = active.reduce(  
    (sum, service) => sum + service.price,  
    0,  
  );  
  
  return {  
    count: active.length,  
    averagePrice: active.length ? totalPrice / active.length : 0,  
  };  
}
```


EXERCÍCIO DE FIXAÇÃO — Miniprojeto

1. Implemente todos os requisitos listados na seção 23.1.
2. Crie ao menos oito serviços de teste no app.js.
3. Provoque intencionalmente três erros e confirme as mensagens produzidas.
4. Refatore qualquer função que ultrapasse uma responsabilidade clara.
5. Faça um commit com mensagem coerente após concluir a primeira versão funcional.

Desafio profissional: Adicione uma busca textual por parte do nome e uma ordenação por price sem modificar o array original.

24 ANTI-PADRÕES QUE DEVEM SER EVITADOS DESDE O INÍCIO

Anti-padrão	Problema	Alternativa
Tudo em app.js	crece sem fronteiras e mistura responsabilidades	extrair módulos conforme surgem responsabilidades
Variáveis globais mutáveis	qualquer parte altera o estado	encapsular estado e expor operações
catch (e) {}	esconde falhas e dificulta diagnóstico	tratar ou propagar conscientemente
== em validações	coerções inesperadas	usar === e conversões explícitas
Número mágico	regra sem significado	constante com nome de domínio
Copiar arquitetura pronta	pastas sem propósito	estrutura incremental orientada a problemas
Depender de pacote para tudo	complexidade e superfície de manutenção	usar APIs nativas quando forem suficientes
Misturar validação e persistência	acoplamento	separar fronteira, regra e dados

25 CHECKLIST DE DOMÍNIO DA SINTAXE

Antes de avançar para HTTP e Fastify, o acadêmico deve conseguir demonstrar os itens abaixo sem depender de copiar um projeto pronto:

- Criar e executar um projeto ESM com Node.js.
- Explicar quando usar const e let.
- Diferenciar null, undefined, falsy e NaN.
- Usar ===, optional chaining e ?? corretamente.
- Escrever funções com parâmetros padrão, rest e arrow functions.
- Usar map, filter, find, some, every e reduce.
- Usar destructuring e spread sem alterar acidentalmente o objeto de origem.
- Converter dados externos com Number e validar o resultado.
- Usar JSON.stringify e JSON.parse com tratamento de erro.
- Criar módulos com import/export.
- Importar APIs nativas com node: quando apropriado.
- Lançar e capturar erros conscientemente.
- Utilizar uma Promise com async/await.

- Ler process.argv e process.env.
- Organizar um caso de uso simples em arquivos separados por responsabilidade.

26 ATIVIDADES DE FIXAÇÃO DO MÓDULO

26.1 Atividade individual — diagnóstico de código

O professor disponibilizará um arquivo contendo código propositalmente ruim. O acadêmico deverá identificar e justificar no mínimo dez problemas relacionados aos conteúdos deste módulo, incluindo nomenclatura, mutação, comparação, validação, organização, tratamento de erro e dependências.

26.2 Atividade prática — refatoração

Refatore o código diagnosticado, mantendo o comportamento esperado. A entrega deve apresentar a estrutura de pastas final, os principais trechos antes/depois e uma justificativa curta para cada decisão relevante.

26.3 Desafio — serviço de agendamento em memória

Implemente uma função `scheduleAppointment` que receba `professionalId`, `serviceId`, `startAt` e `durationMinutes`. A aplicação deve impedir duração inválida e conflito de horário do mesmo profissional. O objetivo ainda não é banco ou HTTP: o foco é decompor a regra em funções compreensíveis e testáveis manualmente.

```
const input = {
  professionalId: "prof-1",
  serviceId: "service-1",
  startAt: "2026-09-10T14:00:00-03:00",
  durationMinutes: 45,
};
```

Questões para discussão após a prática: onde deveria ficar a regra de conflito? O repositório deve decidir se um horário é permitido? O que acontecerá quando existirem duas requisições simultâneas? Quais partes dessa solução mudarão quando o banco PostgreSQL for introduzido?

27 CONSIDERAÇÕES FINAIS

O objetivo deste módulo não foi transformar JavaScript em uma lista de comandos, mas criar uma base comum para raciocinar sobre backend. A partir de agora, cada nova tecnologia terá uma função clara: HTTP resolverá comunicação; Fastify ajudará a estruturar o servidor; schemas resolverão validação crescente; PostgreSQL e migrations resolverão persistência; testes reduzirão regressões; Docker e CI/CD contribuirão para reprodutibilidade e entrega.

A estrutura criada neste ponto é propositalmente pequena. Ela demonstra separação de responsabilidades sem antecipar dezenas de camadas. Conforme o projeto evoluir, o acadêmico

deverá continuar respondendo às perguntas centrais da formação: por que esta abstração existe, qual problema ela resolve, onde a regra pertence, como testaríamos e como isso se comportaria em produção?

28 REFERÊNCIAS

NODE.JS. Node.js v24 Documentation — ECMAScript Modules. Disponível em:

<https://nodejs.org/download/release/latest-v24.x/docs/api/esm.html>. Acesso em: 1 set. 2026.

NODE.JS. Node.js Documentation. Disponível em: <https://nodejs.org/docs/latest-v24.x/api/>. Acesso em: 1 set. 2026.

MDN WEB DOCS. Guia JavaScript. Disponível em:

<https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide>. Acesso em: 1 set. 2026.

MDN WEB DOCS. Módulos JavaScript. Disponível em:

<https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Modules>. Acesso em: 1 set. 2026.

OWASP FOUNDATION. Input Validation Cheat Sheet. Disponível em:

https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html. Acesso em: 1 set. 2026.

OWASP FOUNDATION. Node.js Security Cheat Sheet. Disponível em:

https://cheatsheetseries.owasp.org/cheatsheets/Nodejs_Security_Cheat_Sheet.html. Acesso em: 1 set. 2026.

ECMA INTERNATIONAL. ECMAScript Language Specification. Disponível em:

<https://tc39.es/ecma262/>. Acesso em: 1 set. 2026.